Spark 运行机制理解

第二题：详解 Spark DAGScheduler，贴 Spark Intellij 源码截图加分

DAGScheduler 是 Spark 分布式计算引擎的核心调度组件，它在 SparkContext 初始化时被创建，主要负责将用户提交的计算作业（Job）解析并转换为可执行的任务集合，最终提交给 TaskScheduler 进行分布式执行。**DAGScheduler 的核心功能是实现面向阶段的调度，具体包括以下几个方面：**

DAG 构建：根据 RDD 之间的依赖关系，构建计算任务的有向无环图（DAG）；
Stage 划分：以**宽依赖（ShuffleDependency）**为边界，将 DAG 划分为多个调度阶段（Stage）；
任务集生成：将每个 Stage 转化为由具体任务（Task）组成的任务集（TaskSet），并提交给 TaskScheduler；
执行结果处理：接收 TaskScheduler 返回的任务执行状态（成功/失败），决定是否需要重试或重新提交 Stage。

1. Stage 划分的核心原理
Stage 划分是 DAGScheduler 最核心的职责，其基本原理是逆向递归遍历 RDD 依赖图，以宽依赖为切分点。
1. RDD 依赖关系
RDD 之间的依赖关系分为两种类型：

- 窄依赖（Narrow Dependency）：父 RDD 的每个分区最多被子 RDD 的一个分区使用（如 map、filter 等）。这类操作可以在单个节点上流水线式（pipeline）执行，无需跨节点数据传输；
- 宽依赖（Wide Dependency/ShuffleDependency）：父 RDD 的一个分区可能被子 RDD 的多个分区使用（如 groupByKey、reduceByKey 等）。这类操作需要 Shuffle 过程，涉及跨节点的数据重分布，开销较大。

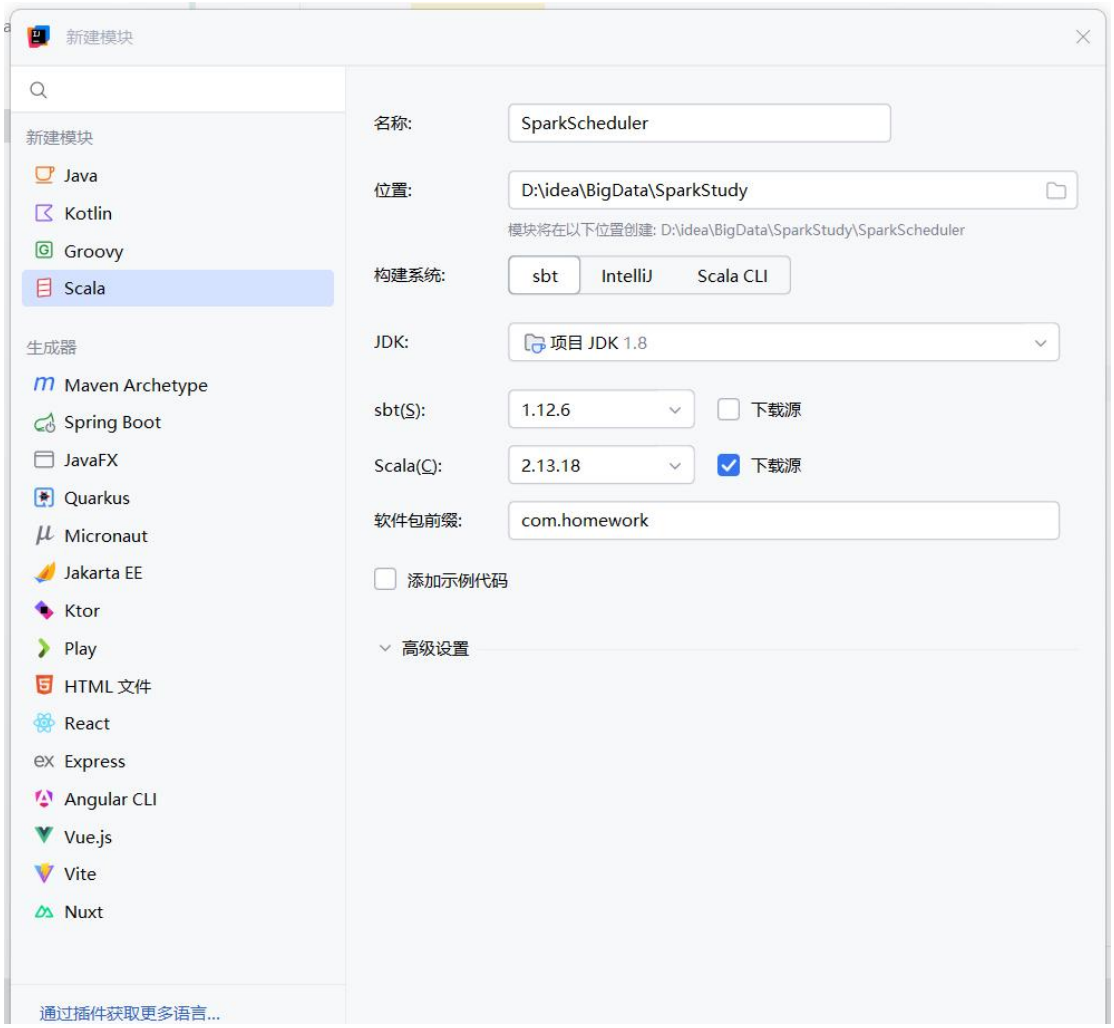
1.2 Stage 类型
根据计算逻辑的不同，Stage 分为两种类型：

- ShuffleMapStage：不是最终的 Stage，其输出需要经过 Shuffle 作为后续 Stage 的输入，对应 ShuffleMapTask；
- ResultStage：最终的 Stage，直接生成作业的最终结果，对应 ResultTask（如 collect、saveAsTextFile 等 Action 操作）。

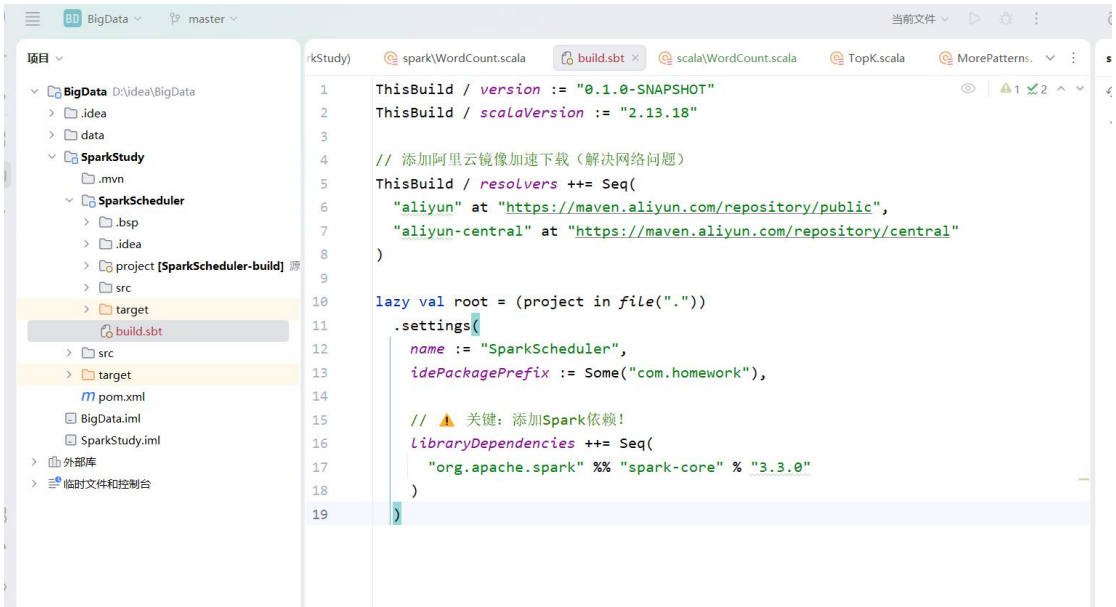
1.3 划分算法
Stage 划分采用逆向递归算法：从触发作业提交的最后一个 RDD（即执行 Action 操作的 RDD）开始，逆向遍历其依赖链。当遇到宽依赖时，即形成一个 Stage 的边界，将宽依赖之前的窄依赖操作划分到同一个 Stage 中，然后以宽依赖的父 RDD 为起点继续向上遍历，直至处理完所有 RDD。
这种算法的优势在于：

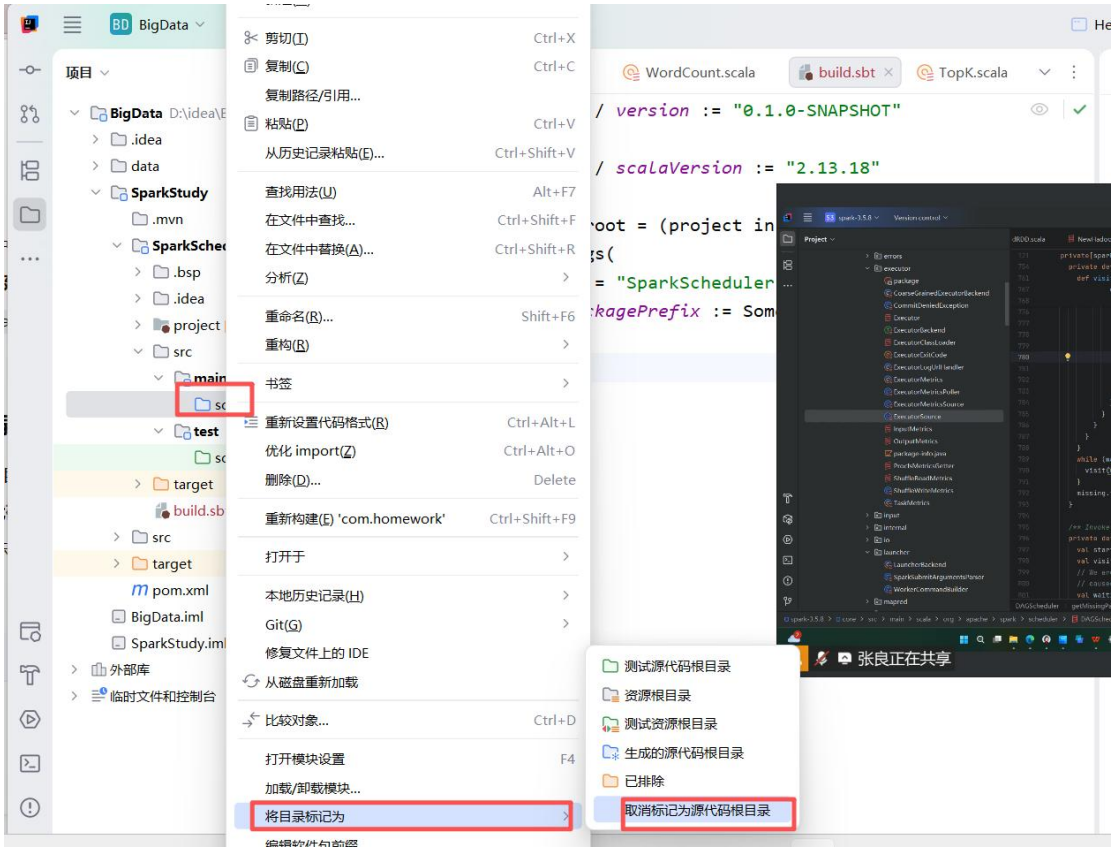
- 保证数据本地性：窄依赖在同一 Stage 内 pipeline 执行，减少数据移动；
- 优化故障恢复：某个 Stage 失败时，只需重新计算该 Stage，无需从头开始；
- 明确执行边界：宽依赖作为 Stage 间的天然分界线，便于 Shuffle 过程的统一管理。

在 IDEA 中创建 Spark 项目：


新建模块
新建模块
Java
Kotlin
Groovy
Scala
生成器
Maven Archetype
Spring Boot
JavaFX
Quarkus
Micronaut
Jakarta EE
Ktor
Play
HTML 文件
React
Express
Angular CLI
Vue.js
Vite
Nuxt
通过插件获取更多语言...
名称:
SparkScheduler
位置:
D:\idea\BigData\SparkStudy
模块将在以下位置创建: D:\idea\BigData\SparkStudy\SparkScheduler
构建系统:
sbt
IntelliJ
Scala CLI
JDK:
项目 JDK 1.8
sbt(S):
1.12.6
下载源
Scala(C):
2.13.18
下载源
软件包前缀:
com.homework
添加示例代码
高级设置

uild.sbt 的配置：


BigData
master
当前文件
项目
BigData D:\idea\BigData
.idea
data
SparkStudy
.mvn
SparkScheduler
.bsp
.idea
project [SparkScheduler-build]
src
target
build.sbt
src
target
pom.xml
BigData.iml
SparkStudy.iml
外部库
临时文件和控制台
spark\WordCount.scala
build.sbt
scala\WordCount.scala
TopK.scala
MorePatterns.
ThisBuild / version := "0.1.0-SNAPSHOT"
ThisBuild / scalaVersion := "2.13.18"
// 添加阿里云镜像加速下载（解决网络问题）
ThisBuild / resolvers ++= Seq(
"aliyun" at "https://maven.aliyun.com/repository/public",
"aliyun-central" at "https://maven.aliyun.com/repository/central"
)
lazy val root = (project in file("."))
.settings(
name := "SparkScheduler",
idePackagePrefix := Some("com.homework"),
// ⚠️ 关键：添加Spark依赖!
libraryDependencies ++= Seq(
"org.apache.spark" %% "spark-core" % "3.3.0"
)
)



运行程序代码：

```
package com.homework

import org.apache.spark.{SparkConf, SparkContext}

object WordCount {
  def main(args: Array[String]): Unit = {
    // 1. 创建 Spark 配置（local 表示本地模式运行）
    val conf = new SparkConf()
      .setAppName("WordCount")
      .setMaster("local[*]")  // local[*] 表示使用本地所有可用
核心

    // 2. 创建 SparkContext（Spark 入口）
    val sc = new SparkContext(conf)
    sc.setLogLevel("ERROR")    // 减少日志输出，只显示错误信息
```

```
    try {
      // 3. 创建测试数据（RDD）
      val data = Seq("Hello Spark", "Hello World", "Spark DAG
Scheduler")
      val rdd = sc.parallelize(data)

      // 4. 转换操作（构建 DAG）
      val words = rdd.flatMap(_.split(" "))
      val wordPairs = words.map(word => (word, 1))
      val wordCounts = wordPairs.reduceByKey(_ + _)

      // 5. 行动操作（触发 Job 提交，DAGScheduler 开始工作）
      val result = wordCounts.collect()

      // 6. 打印结果
      println("=" * 40)
      println("WordCount 结果: ")
      result.foreach { case (word, count) =>
        println(s"$word: $count")
      }
      println("=" * 40)

    } finally {
      // 7. 关闭 SparkContext
      sc.stop()
    }
  }
}
```

```scala
object WordCount {
  def main(args: Array[String]): Unit = {
      .setAppName("WordCount")
      .setMaster("local[*]")  // local[*] 表示使用本地所有可用核心

    // 2. 创建SparkContext（Spark入口）
    val sc = new SparkContext(conf)
    sc.setLogLevel("ERROR")    // 减少日志输出，只显示错误信息
```

```
26/03/21 13:59:55 INFO BlockManagerMasterEndpoint: Registering block manager DESKTOP-KN89BDU:53086 with 4.0 GiB RAM, BlockMan
26/03/21 13:59:55 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, DESKTOP-KN89BDU, 53086, None)
26/03/21 13:59:55 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, DESKTOP-KN89BDU, 53086, None)
========================================
WordCount结果:
Hello: 2
World: 1
DAG: 1
Scheduler: 1
Spark: 2
========================================

进程已结束，退出代码为 0
```

查找 DAGScheduler 源码

```scala
 *  - When adding a new data structure, update `DAGSchedulerSuite.assertDataStructuresEmpty`
 *    include the new structure. This will help to catch memory leaks.
 */
private[spark] class DAGScheduler(
    private[scheduler] val sc: SparkContext,
    private[scheduler] val taskScheduler: TaskScheduler,
    listenerBus: LiveListenerBus,
    mapOutputTracker: MapOutputTrackerMaster,
    blockManagerMaster: BlockManagerMaster,
    env: SparkEnv,
    clock: Clock = new SystemClock())
  extends Logging {

  def this(sc: SparkContext, taskScheduler: TaskScheduler) = {
    this(
      sc,
      taskScheduler,
      sc.listenerBus,
      sc.env.mapOutputTracker.asInstanceOf[MapOutputTrackerMaster],
```

DAGScheduler 核心源码位置

handleJobSubmitted　Job 提交入口

```scala
    listenerBus.post(SparkListenerTaskGettingResult(taskInfo))
  }

  private[scheduler] def handleJobSubmitted(jobId: Int,
      finalRDD: RDD[_],
      func: (TaskContext, Iterator[_]) => _,
      partitions: Array[Int],
      callSite: CallSite,
      listener: JobListener,
      properties: Properties): Unit = {
    var finalStage: ResultStage = null
    try {
      // New stage creation may throw an exception if, for example, jobs are run on a
      // HadoopRDD whose underlying HDFS files have been deleted.
      finalStage = createResultStage(finalRDD, func, partitions, jobId, callSite)
    } catch {
      case e: BarrierJobSlotsNumberCheckFailed =>
        // If jobId doesn't exist in the map, Scala coverts its value null to 0: Int automatically
```

createResultStage　创建最终 Stage

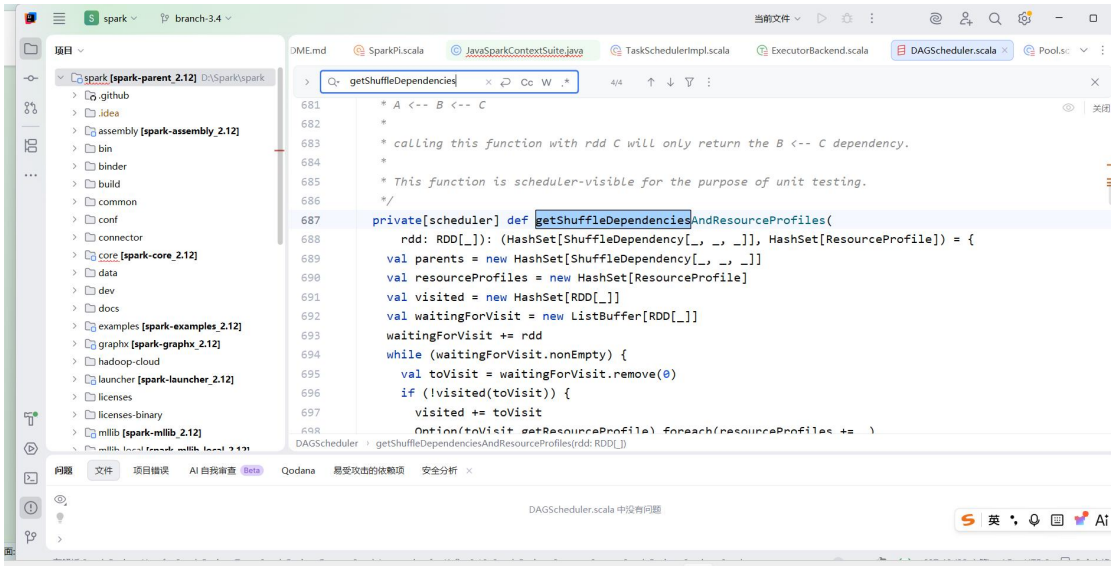
```scala
  /**
   * Create a ResultStage associated with the provided jobId.
   */
  private def createResultStage(
      rdd: RDD[_],
      func: (TaskContext, Iterator[_]) => _,
      partitions: Array[Int],
      jobId: Int,
      callSite: CallSite): ResultStage = {
    val (shuffleDeps, resourceProfiles) = getShuffleDependenciesAndResourceProfiles(rdd)
    val resourceProfile = mergeResourceProfilesForStage(resourceProfiles)
    checkBarrierStageWithDynamicAllocation(rdd)
    checkBarrierStageWithNumSlots(rdd, resourceProfile)
    checkBarrierStageWithRDDChainPattern(rdd, partitions.toSet.size)
    val parents = getOrCreateParentStages(shuffleDeps, jobId)
    val id = nextStageId.getAndIncrement()
    val stage = new ResultStage(id, rdd, func, partitions, parents, jobId,
      callSite, resourceProfile, id)
```

getOrCreateParentStages　获取父 Stage

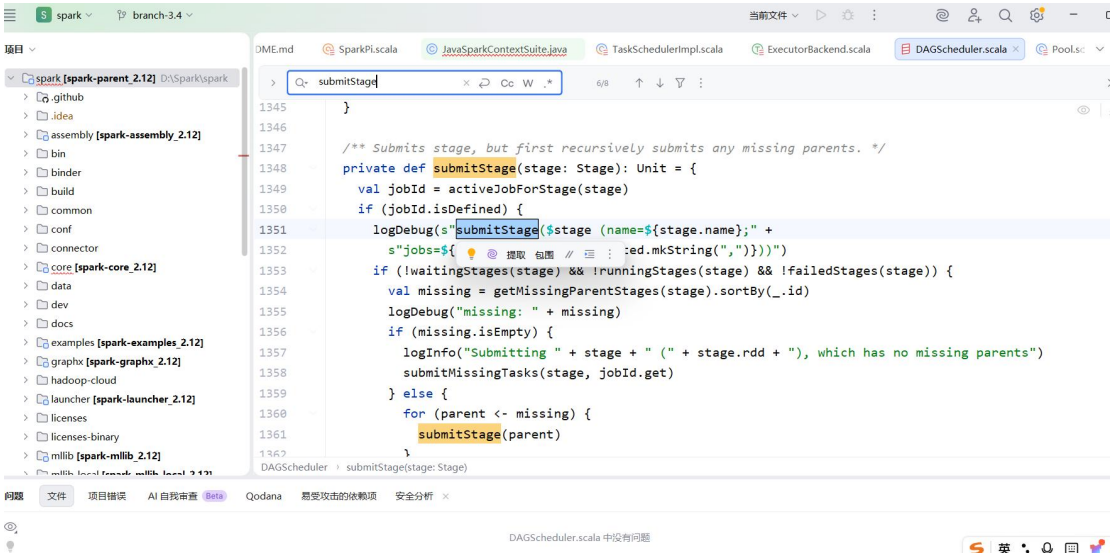
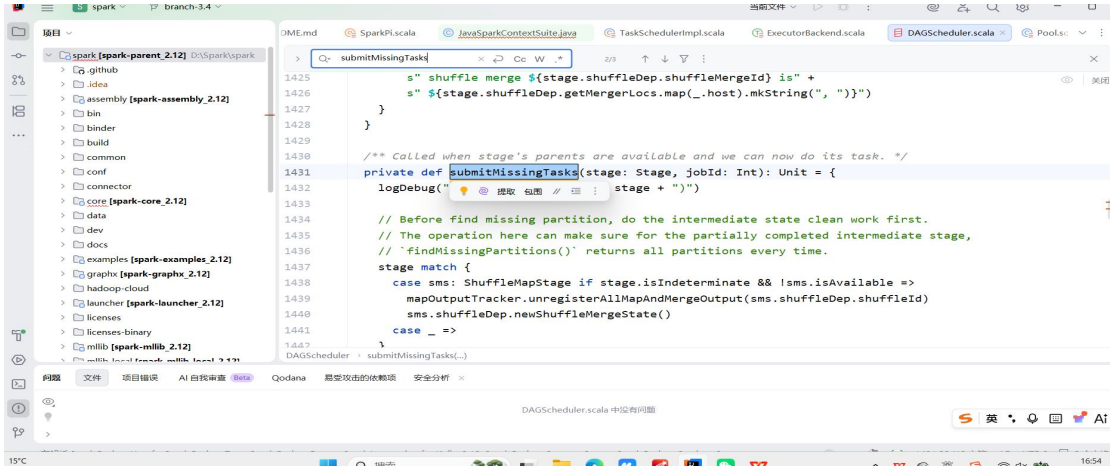
```scala
    * Get or create the list of parent stages for the given shuffle dependencies. The new
    * Stages will be created with the provided firstJobId.
    */
  private def getOrCreateParentStages(shuffleDeps: HashSet[ShuffleDependency[_, _, _]],
      firstJobId: Int): List[Stage] = {
    shuffleDeps.map { shuffleDep =>
      getOrCreateShuffleMapStage(shuffleDep, firstJobId)
    }.toList
  }

  /** Find ancestor shuffle dependencies that are not registered in shuffleToMapStage yet */
  private def getMissingAncestorShuffleDependencies(
      rdd: RDD[_]): ListBuffer[ShuffleDependency[_, _, _]] = {
    val ancestors = new ListBuffer[ShuffleDependency[_, _, _]]
    val visited = new HashSet[RDD[_]]
    // We are manually maintaining a stack here to prevent StackOverflowError
    // caused by recursively visiting
    val waitingForVisit = new ListBuffer[RDD[_]]
```

getShuffleDependencies　　Stage 划分核心算法



submitStage　　提交 Stage



submitMissingTasks　　生成 TaskSet